```
  def apply() = Behaviors.setup[Command] { context =>
    val listingNotificationAdapter =
      context.messageAdapter[Receptionist.Listing](ListingResponse)       ❸

    context.system.receptionist ! Receptionist
      .Subscribe(goldenKey, listingNotificationAdapter)                   ❹

    Behaviors.receiveMessage {
      case ListingResponse(goldenKey.Listing(listings)) =>                ❺
        listings.foreach { actor =>
          context.log.info(s"${actor.path.name} is in")
        }
        Behaviors.same
    }
  }
}
```

❶ The key, attached to the guest protocol

❷ Adapter message to receive the message from the receptionist

❸ Adapter function to translate the message from the receptionist

❹ Subscription to the key

❺ Message from the receptionist

Let’s test that this works as expected. Each time a guest enters the hotel, the list of subscribed guests is sent to each concierge.

Listing 6.2 Using the receptionist

```
import org.scalatest.wordspec.AnyWordSpecLike
import org.scalatest.matchers.should.Matchers

class ReceptionistUsageSpec extends ScalaTestWithActorTestKit
    with AnyWordSpecLike with Matchers with LogCapturing {

  "An actor subscribed to a ServiceKey " should {
    "get notified about all the actors each time an actor registers" in {
      val guest = spawn(VIPGuest(), "Mr.Wick")
      spawn(HotelConcierge())                                             ❶
      LoggingTestKit.info("Mr.Wick is in").expect {
        guest ! VIPGuest.EnterHotel                                       ❷
      }
      val guest2 = spawn(VIPGuest (), "Ms.Perkins ")
      LoggingTestKit.info("Ms.Perkins is in").expect {
        LoggingTestKit.info("Mr.Wick is in").expect {
```